CauseRL: Reinforcement Learning-based Random Walks for Root Cause Analysis in Microservices

Jhon Sebastian Rojas Rodriguez
Université de Lorraine, CNRS, Inria, LORIA
F-54000 Nancy, France
jhon-sebastian.rojas-rodriguez@loria.fr

Abdelkader Lahmadi
Université de Lorraine, CNRS, Inria, LORIA
F-54000 Nancy, France
abdelkader.lahmadi@loria.fr

Michaël Rusinowitch
Université de Lorraine, CNRS, Inria, LORIA
F-54000 Nancy, France
michael.rusinowitch@inria.fr

Abstract

Microservices architecture has become a prevalent design paradigm for Cloud applications. However, the inherent complexity of such architectures significantly complicates the identification of root causes of system failures. This challenge has been the focus of recent research efforts, particularly through the use of causal discovery methods combined with scoring techniques. Existing scoring approaches, however, often explore causal graphs randomly relying only on the structure of the graph, leading to incorrect attribution of importance or missing true root causes. In this paper, we present CauseRL, a Deep Reinforcement Learning-based scoring method designed to more effectively rank potential root cause candidates. Experiments conducted on a recent causal-inference–based Root Cause Analysis benchmark demonstrate that CauseRL achieves improvements up to 5x in performance over traditional methods.

CCS Concepts

• **Software and its engineering → Software reliability**; • **Computing methodologies → Causal reasoning and diagnostics**.

Keywords

Root Cause Analysis, Microservice Systems, Causal Inference, Reinforcement Learning, Random Walks

ACM Reference Format:
Jhon Sebastian Rojas Rodriguez, Abdelkader Lahmadi, and Michaël Rusinowitch. 2026. CauseRL: Reinforcement Learning-based Random Walks for Root Cause Analysis in Microservices. In *The 41st ACM/SIGAPP Symposium on Applied Computing (SAC '26), March 23–27, 2026, Thessaloniki, Greece.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3748522.3779836

1 Introduction

Microservices architectures have emerged as the de facto standard for developing Cloud Applications [23]. Leading industry players such as Amazon, Netflix, X, and Spotify rely on this architectural paradigm to build and operate their core business applications. Being a collection of small, independent, and loosely coupled services enables scalability, flexibility and resilience in cloud-native environments. While beneficial, the distributed nature of microservices introduces a set of potential failure points and new challenges. Network-related failures such as high latencies, timeouts or network partitions as well as communication protocol incompatibilities can lead to performance bottlenecks or service unavailability. The increased complexity of dozens or hundreds of services to deploy, monitor, log, and secure makes difficult the task of identifying the root cause of a failed request. This is compounded by the risk of cascading failures, where an issue in one service can propagate to others.

Monitoring large-scale microservices applications is inherently complex due to scalability challenges. Massive volumes of key performance indicators (KPIs), distributed call traces, and system logs are continuously collected across multiple layers of the system, including the instance, host, and service levels. Root Cause Analysis (RCA) solutions process this data to give a set of possible root cause candidates of failures [22]. Methods focusing on monitored KPIs time series from services face three dimensions of complexity: time series length, system size (number of components in the system) and number of metrics monitored for each component. This complexity necessitates a multi-step approach including steps like Anomaly Detection, Causal Discovery, Topology Graph Generation or Root Cause Candidates Ranking. Most part of proposed solutions rely on a graph to represent the topology of the system or the causal relations between metrics/nodes. They then create a list of candidate root causes using graph-based algorithms such as Random Walks [14, 26], personalized versions of PageRank [27, 29] or classical graph traversal approaches like DFS or BFS [6, 19].

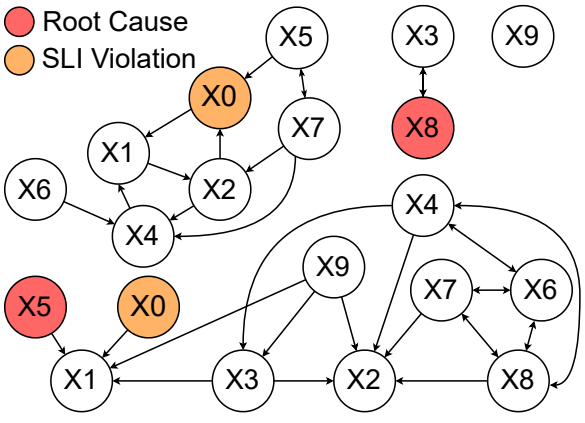


Figure 1: An illustration of two causal graphs from CIRCA10 benchmark. Orange nodes are services where the SLI violation was detected and red nodes are the root cause.

This work is licensed under a Creative Commons Attribution 4.0 International License.
SAC '26, Thessaloniki, Greece
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2294-3/2026/03
https://doi.org/10.1145/3748522.3779836

A recent study has demonstrated that the performance of these widely used methods remains insufficient for practical, real-world applications [18]. They often fail to identify failure's root causes on large datasets (> 200 metrics); this issue might stem from inaccuracies in the causal graph discovery phase. Root cause nodes can appear isolated or their causal relations may not highlight their importance to the failure. Figure 1 illustrates these situations, in both cases the root cause node is not reachable from the initial observed failure X0. In Figure 1 (down) the low input/output degree of node X5 will cause classical scoring methods to attribute more importance to other nodes like X1 or X2. Random walk-based scoring processes that rely only on discovered causal relations might fail to effectively distinguish the relevance of root causes nodes. To overcome these limitations, methods that adapt to misleading causal graphs are needed. Consequently, there is a need to develop more sophisticated techniques to score root cause candidates, leveraging both causal discovered relations and the underlying time series data from monitored metrics.

This paper takes a major step in this direction by introducing a reinforcement learning (RL)-based scoring method that guides and biases the random walk process toward the most relevant nodes in the causal graph. Our approach integrates classical score-based anomaly detection methods to train an RL agent that systematically navigates the causal graph structure, thereby enhancing the robustness and effectiveness of root cause analysis.

The contributions of this work are as follows:

- We propose a novel RCA scoring method that formulates a reinforcement learning task to bias random walk-based scoring techniques toward the most relevant parts of the causal graph [1].
- We developed an environment that implements this formulation based on a recent RCA benchmark [18].
- We evaluated several time series representations techniques in the form of anomaly scores or embeddings.
- We extensively evaluated our approach on eight datasets demonstrating its robustness with a significant performance improvement over existing scoring techniques.

The remainder of this paper is organized as follows. Section 2 presents the problem formulation and key background concepts. Section 3 reviews prior graph-based RCA approaches, categorized by data type and graph representation. Section 4 details the proposed RL-based method. Section 5 outlines the experimental setup, and Section 6 reports and discusses the results. Finally, Section 7 concludes the paper and highlights directions for future work.

2 Background

This section establishes the foundational concepts and techniques inherent in Root Cause Analysis (RCA) and our proposed method.

2.1 Root Cause Analysis

Root Cause Analysis is the problem of identifying the part of the system where a failure started. The analysis involves a comprehensive examination of monitoring data, i.e., metrics data. Monitoring systems record various critical information in the microservice application such as workload, resource consumption and service response times. Typically, these metrics constitute multivariate time series with each time series corresponding to the data collected within a specific service.

In this work, we follow the formulation proposed by [18]. We consider N services $\{s^i\}_{i=1}^N$ composing a large-scale microservice application. At each time step t, M metrics are collected by the monitoring system: $\mathbf{X}_t^i = \{x_t^{(i,j)}\}_{j=1}^M (M \geq 1)$ for each service s^i. When a failure occurs, we denote the metrics collected from before the failure time t_0 until the time when RCA is started t_{rca} as $\mathbf{D} = \{\mathbf{X}_{t_0}^1, ..., \mathbf{X}_{t_{rca}}^N\}$. RCA methods are asked to identify the root cause of the failure from $\mathbf{D}$.

2.2 Anomaly Detection

Anomaly detection is the problem of finding patterns in data that cannot be considered as normal behavior [4]. In microservices applications these anomalies can be symptoms of failures. The detection and understanding of these symptoms is critical to find the underlying root cause of abnormal behaviours. The problem has been extensively studied, including recent work focusing on microservice Cloud applications [22].

In this work we consider mainly distance-based techniques allowing to compute an anomaly score for each KPI's time series. These algorithms detect anomalies by measuring the distance between given data points. Neighborhood methods like k-th Nearest Neighbor (KNN) [5] or Local Outlier Factor(LOF) [2] calculate the distance between a time series instance and its nearest neighbors. Clustering-based approaches leverage a clustering algorithm to calculate distances relative to clusters. Methods like BIRCH [28], DBSCAN [8] and K-Means [12] are common choices.

2.3 Causal Discovery

Causal discovery methods for time series data have gained attention in recent years in microservices applications RCA for their capacity to infer causal relationships between KPIs. These techniques help analysts by modeling complex systems and highlighting direct and indirect causal pathways. The assumption is that these underlying cause-effect relationships are more meaningful than simple correlations, and allows to uncover the true sources of problems rather than merely symptoms. Although in this work we focus on the Peter-Clark (PC) algorithm[24], the most popular and robust for RCA[18], there exists, however other algorithms such as FCI[24], Granger[10], LiNGAM[21] GES[7] and NTLR[9]. The PC algorithm first assumes a fully connected graph and then iteratively tests for conditional independencies between variables to remove edges that lack a direct causal relationship.

2.4 Scoring Methods

Once a causal discovery method has generated a causal graph, it is needed to rank the nodes of the graph. This is typically done using a scoring method. Random walk-based techniques [14, 26] visit the nodes in the causal graph, choosing randomly next node to visit; the nodes that are the most visited are considered as candidates for the root cause of the failure. Similarly, PageRank-based approaches [27, 29] look for the most influential nodes assessing its importance based on the number and quality of incoming edges.

[1] https://gitlab.inria.fr/jrojasro/random_walk_rl

It is possible to add a random teleportation with some probability (dumping factor) to ensure visit disconnected components in the graph. Graph traversal algorithms like Breath First Search (BFS) and Depth First Search (DFS) help identifying paths between services where a failure was found and possible root causes [6]. Finally, Hypothesis Testing methods [29] interpret failures as events that change the pre-failure data distribution. They compute deviation scores, assessing whether the data from a node follow this pre-failure distribution. Nodes with a large deviation are considered to be candidate root causes.

2.5 Reinforcement Learning

Reinforcement learning [25] is a machine learning paradigm inspired by behavioral psychology, particularly by the idea that humans naturally learn by interacting with their environment.

The problem of reinforcement learning can be formalized by a Markov Decision Process, a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$ where $\mathcal{S}$ is a finite set of states, $\mathcal{A}$ is a finite set of actions, $\mathcal{P}$ is a transition probability matrix, $\mathcal{R}$ is a reward function, γ is a discount factor $\in [0, 1]$. A *policy* π defines how the agent behaves at any given time. It is a mapping from observed states to actions chosen by the agent. The reward function R defines the objective of the problem, which is a scalar signal sent by the environment at every time step. In Deep ReDRL, policies are typically represented by a deep neural network, which is trained through interaction with the environment to discover strategies that lead to successful long-term outcomes.

To solve a reinforcement learning problem, the goal is to find the policy that successfully optimizes long-term rewards. Starting from an estimation of a policy, it is improved by slightly increasing the probability of reaching states with higher rewards. This is known as Policy Iteration. One of the most successful policy-based algorithms is Proximal Policy Optimization (PPO) [20].

When the state of the environment is not directly observable, the partial observable MDP model (POMDP) generalizes the MDP by taking into account more forms of uncertainty in the environment. POMDP introduces a finite set of observations $\mathcal{Z}$ and an observation function $O : \mathcal{S}x\mathcal{A} \rightarrow \pi(\mathcal{Z})$, where $\pi()$ is a probability distribution over some finite set. The observation set consists of all the possible observations that the environment can emit.

3 Related Work

Root cause analysis causality graph-based solutions [6, 14, 19, 26, 15, 16] find the root cause by deriving automatically and visiting a causality graph. Typically the vertices in the graph model services, with each oriented edge indicating a dependency between the source service and the target service. Causal discovery algorithms as Peter-Clark (PC) [24], Fast Causal Inference (FCI) [24], Granger [10], LiNGAM [21], Greedy Equivalence Search (GES) [7] or NOTEARS-Low-Rank (NTLR) [9] are typically used to derive the causality graph and the common visiting methods are random walks and BFS.

CauseInfer [6] and Microscope [14] use PC algorithm to create the causal graph and enrich it with additional edges corresponding to service call dependencies. Both methods create dependency chains of services visiting recursively the obtained graph starting from the frontend application, for each node visited they consider the services on which it depends and if none of them is considered anomalous the current node is a candidate root cause. CauseInfer uses cumulative sum statistics of monitored response times to compute anomaly scores for each service and use this score to rank the root cause candidates. Microscope uses the Pearson correlation between candidates and the frontend application service as ranking score.

MicroDiag[27] applies PageRank to a LiNGAM generated causal graph to rank the root causes candidates. CausalRCA[29] the causal structure learning DAG-GNN method to generate a causal graph. This is a gradient-based variational autoencoder. Then it uses PageRank to infer the root cause.

Qiu et al [19] aim to identify root causes from any failing service in the system. The nodes of the causality graph are monitored KPIs from the services. They complement the graph adding edges between KPIs whenever there is a dependency between services. Pearson correlation is used to weight the edges and BFS is used to construct all possible paths from the origin node to root cause candidates. The paths are sorted based on the sum of the weights of its visited nodes and prioritizing shortest path if there is multiple candidates with the same sum.

Other methods like CloudRanger [26] MS-Rank[15] and AutoMap [16] use Random Walks to score causality graphs generated by the PC algorithm. In contrast to CauseInfer and Microscope, they use the d-separation condition independence test in the construction of the causality graph. The random walk starts from the frontend service and for a fixed number of iterations they visit a node from the neighborhood of the currently visited service. The ranking is computed based on the number of times services have been visited.

CIRCA[13] derives a causal graph from the application's call graph by leveraging operational expertise to map metrics into predefined categories. It conceptualizes a failure as an intervention that shifts the underlying distribution of metric data. Root cause identification is subsequently performed via a regression-based hypothesis test against the constructed causal model.

CausalAI[1] is an open-source library developed by SalesForce for causal analysis in industrial settings. Its RCA methodology involves learning a causal graph from time series metrics via the PC algorithm. The root nodes of the resultant graph are subsequently inferred as the potential root causes of an incident.

To summarize, it does not exists a method leveraging metrics data to guide the scoring process, we took inspiration from this fact to propose our method. Our scoring method based on Deep Reinforcement Learning (DRL) can be used to rank root candidate nodes from a causal graph generated by any causal discovery algorithm.

4 Root Cause Analysis via Reinforcement Learning

In this section we describe a new scoring method based on Deep Reinforcement Learning and Random Walks. Our method improves random walks by training an agent who can choose smartly the next neighbor node to visit. Causal discovery methods might fail to capture the real causal relations between metrics. Our main motivation to use DRL is to train agents not only to recognize real

patterns of failures but also to adapt to misleading causal graphs [18].

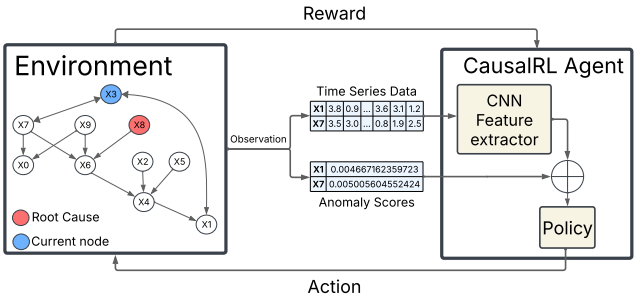


Figure 2: Overall workflow of our CauseRL method by using either anomaly score or CNN-based feature extraction.

We formulate the scoring problem as a POMDP where an agent traverses causality graphs and makes choices regarding the next node to visit in order to maximize a reward feedback based on common RCA performance metrics. The idea is to concentrate the random walk process on potential root cause candidates neighborhoods.

4.1 Learning Environment

To train RL agents for root cause analysis, we developed a Gym-like environment [3] based on datasets from the RCAEval benchmark [18]. Following the benchmark's findings that the PC algorithm is the most reliable causal discovery method, we generated causal graphs for all failure cases in the datasets. In these graphs, nodes represent metrics, and edges represent causal relations between them.

Each fixed-length episode begins by selecting a random failure case from the dataset and placing the agent at a random node in the corresponding causal graph. A random walk process is then initiated, incorporating random teleportations (see [17]) with a damping factor set to 0.15. At each time step without a teleportation event, the agent must select a neighboring node from its current position. We allow going in both directions for each causal relation, as the benchmark found that causal discovery methods have difficulties estimating edge directions. If a node is isolated, the agent remains there until a teleportation occurs. The observation provided to the agent for decision-making includes information about its reachable neighbor nodes. Upon selecting a node, the agent moves to it and the node's visit counter is incremented.

The agent receives a reward signal based on the improvement in the root cause ranking compared to the previous state. After a fixed number of steps, the episode terminates, and the final root cause ranking is determined by the nodes' visit counts.

4.2 Observation and State Space

The state s of the environment consists of the causal graph corresponding to the case being analysed with node visit counters and the current position of the agent. The state space $\mathcal{S}$ then is all the combinations of causal graphs from the dataset and the possible node counters for a fixed-length episode. The agent does not see the complete state of the environment, it gets an observation o with some information about current node's neighbors. In this formulation the observation function O is deterministic and we propose two types of observation spaces $\mathcal{Z}$ for our environment:

- *Anomaly Scores*: We give the agent an anomaly score for each metric. The anomaly score is pre-computed using one of the distance-based anomaly detection techniques discussed in Section 2.2: BIRCH, KNN or Kmeans.
- *Down-sampled Time Series*: The time series for each neighbor is down-sampled in order to fit in memory, as training batches can get large when using the complete signal. The down-sampling uses simple max aggregation given a down-sampling factor.

4.3 Action Space

The action space $\mathcal{A} = \{0, ..., N\}$ is discrete, bounded by N, the maximum out degree found in the causal graphs from the dataset. The selected action a corresponds to moving to the neighbor with index a in the adjacency list of the current node. Since nodes can have different number of neighbors, there are valid and invalid actions at each interaction. To address this, we use the invalid action masking technique [11]. We avoid the action of not moving from the current node to avoid reward hacking behaviours.

4.4 Reward

The reward function $\mathcal{R}$ is a linear combination of performance metrics commonly used in RCA. We adopt two metrics from [18], $AC@k$ and $Avg@k$. The term $AC@k$ models the probability that the root cause is in the top k candidates. Given a set of failure cases C, $AC@k$ is defined as follows,

$$AC@k = \frac{1}{|C|} \sum_{c \in C} \frac{\sum_{i<k} R^c[i] \in V_{rc}^c}{min(k, |V_{rc}^c|)} \tag{1}$$

where $R^c[i]$ is the ranking result for the failure case c and V_{rc}^c is the set of root causes of c. The term $Avg@k$ measures the overall performance of RCA methods and is defined as

$$Avg@k = \frac{1}{k} \sum_{i \le j \le k} AC@j \tag{2}$$

We use $AC@k$ and $Avg@k$ with $k \in \{1, 2, 3, 4, 5\}$ at metrics and service level to compose our reward function. We assign a weight w_k to control the degree of importance we give to metrics with first-top k candidates. We define the rewards as follows:

$$R = \sum_{k=1}^{5} w_k (\Delta AC@k + \Delta Avg@k) \tag{3}$$

where

$$\Delta AC@k = AC@k(s') - AC@k(s)$$

$$\Delta Avg@k = Avg@k(s') - Avg@k(s)$$

with s' being the new state after applying action a selected by the agent and s being the previous state. Respectively, $AC@k(s')$ and $Avg@k(s')$ are the metrics computed for the root cause candidates ranking given by state s' and $AC@k(s)$ and $Avg@k(s)$ for state s. We compute this term also at service level, i.e., the ranking and

metrics are computed from solely the service they belong to. Then the final reward is

$$\mathcal{R} = R_{metric} + R_{service} \tag{4}$$

4.5 RL Policy Design

The input of the policy is the concatenation of the information of the next valid nodes based on the type of environment the agent is interacting with. In the case of the anomaly score-based environments, we concatenate the anomaly scores and pad invalid neighbors up to the fixed amount of maximum neighbors N (see Section 4.3). The variants of CauseRL are denoted as CauseRL-BIRCH, CauseRL-KNN, and CauseRL-KMeans, depending on the underlying anomaly detection technique employed. In the other case, we add to the architecture a one dimension convolution neural network (CNN) feature extractor and concatenate the embeddings of the valid neighbors. The CauseRL variant that incorporates this technique is referred to as CauseRL-CNN. Figure 2 summarizes the workflow of our method.

5 Experimental Setup

Table 1: Description of datasets used in the Study [18]

Name	Nodes	Edges	Cases
CIRCA10	10	20	200
CIRCA50	50	100	200
RCD10	10	13-19	200
RCD50	50	85-104	200

(a) Synthetic datasets

Name	Metrics	Services	Fault Types	Cases
Sock Shop 1	38	13	2	50
Sock Shop 2	46	15	5	125
Online Boutique	49	12	5	125
Train Ticket	212	64	5	125

(b) Real-world datasets

We use a neural network to parameterize our policy π and we train it using PPO. For our CNN feature extractor, we stack several layers of one dimension convolutional neural networks then we use a global average pooling to handle variable-length inputs, and finally we use a fully connected layer to project the output to a fixed-size embedding.

We conducted experiments using the RCAEval benchmark [18]. This allows us to compare the performance of our method with other causal inference-based RCA methods over several datasets widely used for evaluating RCA methods. The benchmark includes both auto-generated synthetic datasets and popular microservice systems benchmarks.

Benchmark datasets (Table 1b) include several cases of different common failure types (Packet loss, Input-Output disk stress, network delay and CPU stress) injected in different services within the application.

Synthetic datasets (Table 1a) introduce faults by changing the conditional probability distribution of a node.

In order to analyze the performance of our method, we formulate the following research questions:

RQ1: Can CauseRL generalize to unseen failure cases?

RQ2: Is CauseRL more efficient that classic Random Walk scoring approaches in terms of number of steps?

RQ3: Is the 1D Convolutional Feature Extractor more effective than anomaly scores for enabling RL-Based root cause analysis?

RQ4: Can CauseRL outperform state-of-the-art causal inference-based methods regarding root cause localization accuracy?

To answer RQ1, i.e., to test the generalization power of our method, we split all the datasets into training and test sets. For benchmark datasets, training sets include 60% of the cases for each failure type, while for synthetic datasets we use 70% of the cases. We train a PPO agent in an environment based on each dataset. We compare the performance on train and test sets using $AC@k$ and $Avg@k$ with $k \in 1, 3, 5$.

To answer RQ2 we compare different walk lengths with CauseRL and PC-RandomWalk/CLoudRanger. We set the number of steps to $n \in \{10, 50, 100, 500, 1000, 5000, 10000, 50000\}$ and compare $Avg@5$ for all methods. We do not consider PageRank methods (as CausalRCA and MicroDiag) since each of its iterations operates on the *entire* graph using matrix operations to compute a stable probability distribution over all nodes, unlike a Random Walk process that simulates a single path through the graph. Independence test-based algorithms (CIRCA and CausalAI) are also excluded because they score each node of the graph independently.

To answer RQ3 we compare the performance of CauseRL for different representations of time series, i.e., anomaly scores obtained using BIRCH, KNN and K-means algorithms and using a one dimension CNN.

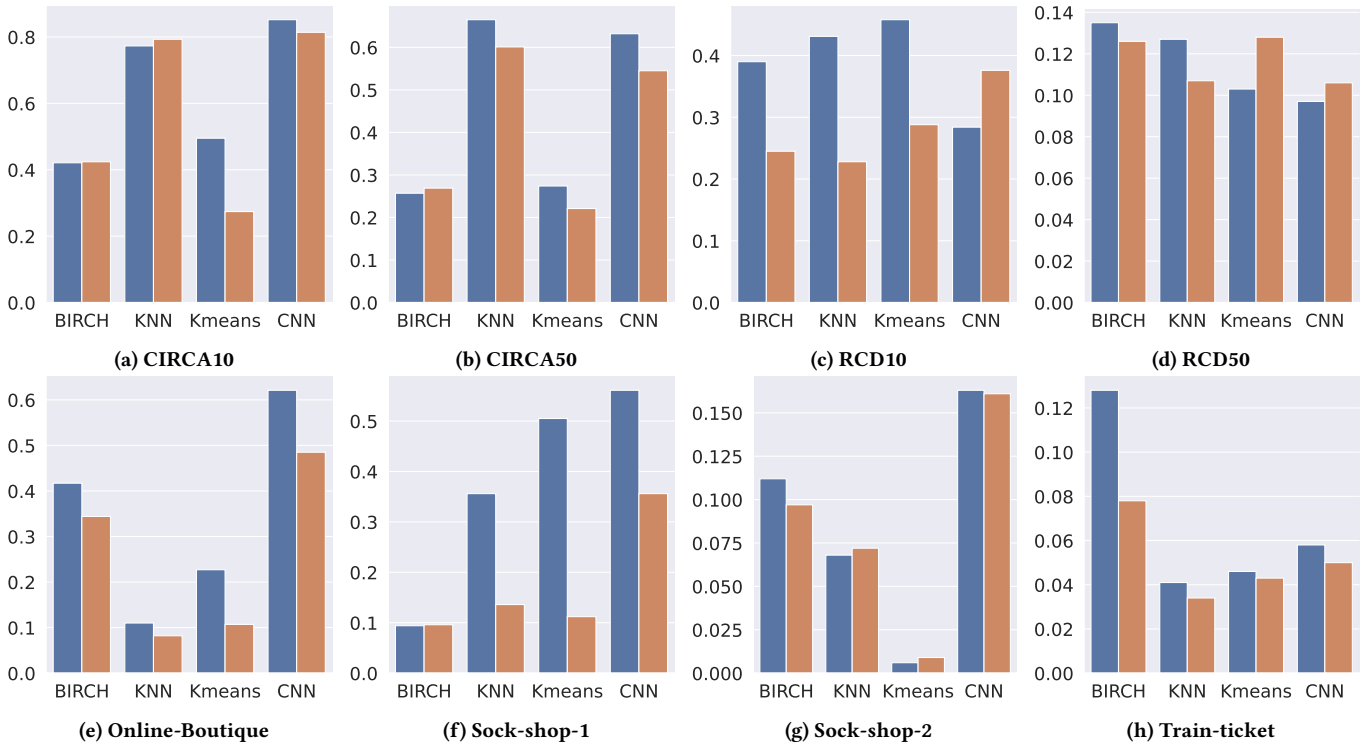
Finally, to answer RQ4 we conducted experiments using RCAEval to compare the performance of state-of-the-art methods with CauseRL in the same test set. We use as baseline nine algorithms provided by the RCAEval benchmark. The dummy agent introduced in [18] is a method that randomly chooses a node within the causal graph as the root cause. CIRCA [13], CloudRanger [26], MicroDiag [27], CausalRCA [29], CausalAI [1] and classic RandomWalk and PageRank with a PC causal learned graph. We compare the performance with $Avg@5$ metric as it measures the overall performance of RCA methods.

6 Results and Discussion

This section presents the experimental results and discusses their implications in direct response to the formulated research questions. We mainly evaluate the proposed reinforcement learning-based RCA method by examining its performance against state-of-the-art techniques, its robustness across diverse datasets, and the impact of different anomaly detection strategies.

6.1 Generalization Study (RQ1)

Figure 3 evaluates the generalization capability of different CauseRL variants by comparing their performance on training and test sets across all benchmark datasets. The results demonstrate that our



(a) CIRCA10 (b) CIRCA50 (c) RCD10 (d) RCD50

(e) Online-Boutique (f) Sock-shop-1 (g) Sock-shop-2 (h) Train-ticket

Figure 3: Avg@5 for train (blue) and test (orange) splits with synthetic (a,b,c,d) and real world application datasets (e,f,g,h).

method generalizes effectively to unseen failure cases in both synthetic and real-world environments.

Analysis of the variants reveals distinct generalization profiles:

CauseRL-BIRCH exhibits the a stable generalization, with an average drop of 13.29% (*Avg*@5) across the all datasets, this model consistently align test performance closely with training performance except in Train Ticket and RCD10 datasets.

CauseRL-Kmeans and **CauseRL-KNN** show the highest volatility, with an average drop of 20.5% and 21.04% respectively. We can see significant performance gaps between training and test sets, particularly on the Sock-shop-1 and RCD10 dataset.

CauseRL-CNN achieves the best overall performance across all datasets with an average drop of 6.25%. In the RCD dataset we observe that the model's performance on the test set slightly exceeds the one of the training set. This suggests that the training examples were sufficiently representative and the test set contains fewer edge cases than the training data. Its superior average accuracy makes it the most effective variant of the proposed methods.

These findings confirm that CauseRL can perform consistently on new failure scenarios even when trained with limited examples. The key to this generalization is that our agent learns a general policy for traversing causal graphs based on underlying anomaly patterns—rather than memorizing specific failures.

6.2 Walk Length Study (RQ2)

Figure 4 illustrates the behaviour of *Avg*@5 for different walk lengths. The CauseRL-CNN model consistently benefits from larger walks while anomaly score-based models seem to have a more unpredictable behavior across different datasets. Increasing the walk length led to performance gains ranging from 30% to 172%. Random Walk and CloudRanger do not exhibit any performance improvement as the walk length increases.

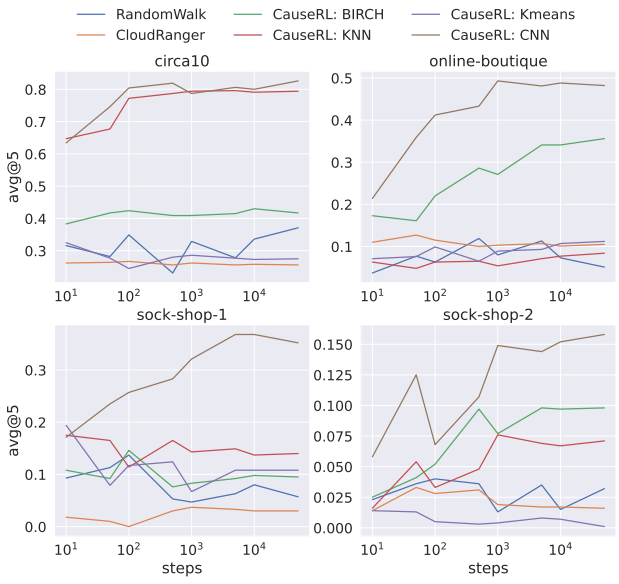


Figure 4: Random walk length ablation study.

Table 2: Performance Comparison of RCA Methods using the Average *Avg*@5 metric over 10 runs

Method	RCD10	RCD50	CIRCA10	CIRCA50	Online Boutique	Sock Shop 1	Sock Shop 2	Train Ticket
Dummy [18]	0.284 ± 0.059	0.07 ± 0.046	0.348 ± 0.065	0.042 ± 0.029	0.031 ± 0.018	0.087 ± 0.031	0.019 ± 0.012	0.009 ± 0.011
PC-PR [18]	0.347 ± 0.0	0.107 ± 0.0	0.433 ± 0.0	0.087 ± 0.0	0.096 ± 0.0	0.12 ± 0.0	0.024 ± 0.0	0.005 ± 0.0
PC-RW [18]	0.286 ± 0.083	0.077 ± 0.065	0.299 ± 0.032	0.053 ± 0.03	0.062 ± 0.033	0.022 ± 0.003	0.016 ± 0.003	0.001 ± 0.002
CausalRCA [29]	0.1 ± 0.0	0.04 ± 0.0	0.161 ± 0.076	0.0 ± 0.0	0.375 ± 0.015	0.441 ± 0.061	0.134 ± 0.026	0,112 ± 0,016
CausalAI [1]	0.278 ± 0.0	**0.133 ± 0.0**	0.0 ± 0.0	0.0 ± 0.0	0.164 ± 0.0	0.03 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
CIRCA [13]	0.356 ± 0.0	0.073 ± 0.0	0.344 ± 0.0	0.053 ± 0.0	0.38 ± 0.0	**0.65 ± 0.0**	0.096 ± 0.0	**0.216 ± 0.0**
MicroDiag [27]	0.026 ± 0.0	0.044 ± 0.007	0.029 ± 0.001	0.046 ± 0.006	0.014 ± 0.004	0.034 ± 0.003	0.018 ± 0.004	0.007 ± 0.006
CloudRanger [26]	0.351 ± 0.011	0.066 ± 0.012	0.255 ± 0.009	0.015 ± 0.011	0.108 ± 0.01	0.021 ± 0.014	0.023 ± 0.008	0.007 ± 0.006
CauseRL-Birch	0.24 ± 0.015	0.126 ± 0.009	0.424 ± 0.011	0.269 ± 0.012	0.344 ± 0.013	0.096 ± 0.007	0.097 ± 0.004	0.078 ± 0.007
CauseRL-Knn	0.228 ± 0.017	0.107 ± 0.003	0.793 ± 0.002	**0.601 ± 0.015**	0.082 ± 0.01	0.136 ± 0.007	0.072 ± 0.008	0.034 ± 0.009
CauseRL-Kmeans	0.288 ± 0.017	0.128 ± 0.014	0.274 ± 0.006	0.221 ± 0.007	0.107 ± 0.009	0.112 ± 0.01	0.009 ± 0.007	0.043 ± 0.014
CauseRL-CNN	**0.376 ± 0.031**	0.106 ± 0.014	**0.814 ± 0.009**	0.545 ± 0.017	**0.485 ± 0.011**	0.356 ± 0.021	**0.161 ± 0.010**	0.05 ± 0.012

Our ablation study on walk length reveals a critical insight: our model's performance improves with longer walks, demonstrating a learned, purposeful navigation strategy. While longer walks provide more opportunity for random exploration, the static performance of baseline models with increased steps highlights their lack of a guiding policy, causing them to drift aimlessly or become misled by irrelevant causality connections.

Unlike baseline models that show no corresponding performance gain, our CauseRL agent models leverage the additional steps to purposefully explore and converge on the most relevant anomalous nodes. This indicates that the agent is not merely wandering but executing an driven, biased search for root causes.

6.3 Anomaly Score vs. Feature Extraction (RQ3)

Table 2 summarizes the performance of all evaluated methods. The results indicate that anomaly score-based representations for CauseRL lead to fluctuating performance across the benchmark. While competitive on some datasets, their effectiveness is inconsistent. Notably, the CauseRL-KNN variant achieves the top score on the CIRCA50 dataset, demonstrating the potential of anomaly-driven approaches in specific scenarios. In contrast, the CauseRL-CNN model delivers the most robust performance, obtaining the highest *Avg*@5 score in four out of the eight benchmark datasets while maintaining competitive results on the remaining four. This consistency establishes CauseRL-CNN as the most reliable variant of our proposed approach.

The performance disparity between feature extraction methods can be explained by their fundamental objectives. Unsupervised anomaly detectors (BIRCH, KNN, K-Means) are task-agnostic; they are designed to identify global point anomalies by measuring deviation from a learned "normal" state. However, a high anomaly score does not necessarily correlate with a root cause, as it could flag an effect or an unrelated outlier. In contrast, the 1D-CNN feature extractor is trained end-to-end within the RL framework. It learns local temporal features (such as sharp spikes, gradual ramps, or period disruptions) that are predictive of a service's role in a failure chain. Consequently, the CNN is explicitly optimized to extract temporally relevant patterns that are most useful for the agent's graph navigation policy, resulting in a representation that is not merely anomalous but causally relevant.

6.4 Performance Comparison (RQ4)

To address RQ4, we compare our method's performance against the state-of-the-art (SOTA) baselines from Table 2. The – indicates that the method timed-out a 24 hours running time limit. CauseRL outperforms the best-performing baselines on five among the eight benchmark datasets: RCD10, CIRCA10, CIRCA50, Online Boutique, and Sock Shop 2, achieving relative improvements ranging from 0.7x to 5.9x over best performance solutions. Furthermore, while the SOTA and classic methods exhibited high performance variance across different benchmarks, CauseRL maintained consistent performance on both synthetic and real-world datasets. This demonstrates our method's robustness and its ability to learn effective navigation policies adaptable to diverse problem settings. We attribute this robustness to our model's capacity to leverage the rich, node-level feature representations, rather than relying solely on the potentially incomplete or misleading causal graph structure used by other methods. Notably, on the Sock Shop 1 and Train Ticket datasets, the specialized CIRCA benchmark achieved superior results. We hypothesize this is because CIRCA's causal model is constructed with operational expertise (e.g., pre-categorized metrics and application call graphs), yielding a more accurate underlying structure. In contrast, our method uses a PC-learned graph without such expert input. Despite this disadvantage, CauseRL achieved competitive and consistent performance

6.5 Limitations and threads to validity

When evaluating the proposed method, several key threats to validity must be acknowledged. A primary concern is reproducibility: RL algorithms are highly sensitive to randomness in initialization, exploration, and environmental stochasticity, making it difficult to ensure consistent, replicable outcomes—even with fixed hyperparameters. Additionally, the method's reliance on deep reinforcement learning (DRL) introduces substantial computational demands. Training the CNN-based model is resource-intensive, often necessitating parallelized batch backpropagation across multiple CPUs and GPUs. Moreover, processing large-scale metrics datasets requires significant memory allocation, especially when running multiple environments concurrently. These computational constraints restrict rigorous hyperparameter tuning and comprehensive sensitivity

analyses. Consequently, the generalizability of our findings to more complex, real-world operational settings remains a challenge.

7 Conclusions and Future Work

In this paper, we have introduced CauseRL, a novel scoring method for RCA that leverages Reinforcement Learning to navigate causal graphs and enhance root cause scoring in microservices architectures. CauseRL generalizes effectively to unseen failure cases because its reasoning is based on learned, abstract patterns of anomalies within causal graphs. Furthermore, the agent adapts its search strategy, it learns to prioritize causal pathways that are currently exhibiting anomalous behavior, making it inherently responsive to the specific context of each failure.

Our walk length ablation provides concrete evidence that the RL agent is learning a meaningful policy: CauseRL exhibits a deliberate, focused exploration which allows to leverage a longer horizon for more effective investigation. In contrast to baseline models that exhibit no coherent strategy, leading to static outcomes.

CNN embeddings provide a rich, adaptive, and RCA-oriented feature representation for time series. Thereby enabling our RL agent to perform a more intelligent and consistent analysis of the causal graph. CauseRL outperforms state-of-the-art baselines on 5 out of 8 datasets, achieving performance gains ranging from 0.7x to 5.9x, while demonstrating superior robustness.

Future work will explore advanced feature extractors, including higher-dimension Convolutional Networks, Recurrent Neural Networks, and Transformers. And policies combining both extracted features and anomaly scores as inputs. We also plan to extend our evaluation with topology graph representations and applications across the entire Computing Continuum.

Acknowledgments

This work is partially funded by a ANR - French government grant under the France 2030 program, project SPIREC of PEPR Cloud (ANR-23-PECL-0006).

References

[1] Devansh Arpit et al. 2023. Salesforce causalai library: a fast and scalable framework for causal analysis of time series and tabular data. (2023). https://arxiv.org/abs/2301.10859 arXiv: 2301.10859 [cs.LG].

[2] Markus M. Breunig, Hans-Peter Kriegel, Raymond T. Ng, and Jörg Sander. 2000. Lof: identifying density-based local outliers. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data* (SIGMOD '00). Association for Computing Machinery, Dallas, Texas, USA, 93–104. ISBN: 1581132174. doi:10.1145/342009.335388.

[3] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. 2016. Openai gym. (2016). https://arxiv.org/abs/1606.01540 arXiv: 1606.01540 [cs.LG].

[4] Varun Chandola, Arindam Banerjee, and Vipin Kumar. 2009. Anomaly detection: a survey. *ACM Comput. Surv.*, 41, (July 2009). doi:10.1145/1541880.1541882.

[5] Wanpracha Chaovalitwongse, Ya Ju Fan, and Rajesh Sachdeo. 2007. On the time series k-nearest neighbor classification of abnormal brain activity. *Systems, Man and Cybernetics, Part A: Systems and Humans, IEEE Transactions on*, 37, (Dec. 2007), 1005–1016. doi:10.1109/TSMCA.2007.897589.

[6] Pengfei Chen, Yong Qi, and Di Hou. 2019. Causeinfer: automated end-to-end performance diagnosis with hierarchical causality graph in cloud environment. *IEEE Transactions on Services Computing*, 12, 2, 214–230. doi:10.1109/TSC.2016.2607739.

[7] David Maxwell Chickering. 2002. Learning equivalence classes of bayesian-network structures. *J. Mach. Learn. Res.*, 2, (Mar. 2002), 445–498. doi:10.1162/153244302760200696.

[8] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. 1996. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining* (KDD'96). AAAI Press, Portland, Oregon, 226–231.

[9] Zhuangyan Fang, Shengyu Zhu, Jiji Zhang, Yue Liu, Zhitang Chen, and Yangbo He. 2023. On low rank directed acyclic graphs and causal structure learning. (2023). https://arxiv.org/abs/2006.05691 arXiv: 2006.05691 [cs.LG].

[10] C.W.J. Granger. 1980. Testing for causality: a personal viewpoint. *Journal of Economic Dynamics and Control*, 2, 329–352. doi:https://doi.org/10.1016/0165-1889(80)90069-X.

[11] Shengyi Huang and Santiago Ontañón. 2022. A closer look at invalid action masking in policy gradient algorithms. *The International FLAIRS Conference Proceedings*, 35, (May 2022). doi:10.32473/flairs.v35i.130584.

[12] István Kiss, Béla Genge, Piroska Haller, and Gheorghe Sebestyén. 2014. Data clustering-based anomaly detection in industrial control systems. In *2014 IEEE 10th International Conference on Intelligent Computer Communication and Processing (ICCP)*, 275–281. doi:10.1109/ICCP.2014.6937009.

[13] Mingjie Li, Zeyan Li, Kanglin Yin, Xiaohui Nie, Wenchi Zhang, Kaixin Sui, and Dan Pei. 2022. Causal inference-based root cause analysis for online service systems with intervention recognition. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (KDD '22). ACM, (Aug. 2022), 3230–3240. doi:10.1145/3534678.3539041.

[14] Jinjin Lin, Pengfei Chen, and Zibin Zheng. 2018. Microscope: pinpoint performance issues with causal graphs in micro-service environments. In *Service-Oriented Computing*. Claus Pahl, Maja Vukovic, Jianwei Yin, and Qi Yu, (Eds.) Springer International Publishing, Cham, 3–20. ISBN: 978-3-030-03596-9.

[15] Meng Ma, Weilan Lin, Disheng Pan, and Ping Wang. 2019. Ms-rank: multi-metric and self-adaptive root cause diagnosis for microservice applications. In *2019 IEEE International Conference on Web Services (ICWS)*, 60–67. doi:10.1109/ICWS.2019.00022.

[16] Meng Ma, Jingmin Xu, Yuan Wang, Pengfei Chen, Zonghua Zhang, and Ping Wang. 2020. Automap: diagnose your microservice-based web applications automatically. In *Proceedings of The Web Conference 2020* (WWW '20). Association for Computing Machinery, Taipei, Taiwan, 246–258. ISBN: 9781450370233. doi:10.1145/3366423.3380111.

[17] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. 1999. The PageRank Citation Ranking: Bringing Order to the Web. Technical Report 1999-66. Previous number = SIDL-WP-1999-0120. Stanford InfoLab, (Nov. 1999). http://ilpubs.stanford.edu:8090/422/.

[18] Luan Pham, Hongyu Zhang, Huong Ha, Flora Salim, and Xiuzhen Zhang. 2025. Rcaeval: a benchmark for root cause analysis of microservice systems with telemetry data. (2025). https://arxiv.org/abs/2412.17015.

[19] Juan Qiu, Qingfeng Du, Kanglin Yin, Shuang-Li Zhang, and Chongshu Qian. 2020. A causality mining and knowledge graph based method of root cause diagnosis for performance anomaly in cloud applications. *Applied Sciences*, 10, (Mar. 2020), 2166. doi:10.3390/app10062166.

[20] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. (2017). https://arxiv.org/abs/1707.06347 arXiv: 1707.06347 [cs.LG].

[21] Shohei Shimizu, Patrik O. Hoyer, Aapo Hyvärinen, and Antti Kerminen. 2006. A linear non-gaussian acyclic model for causal discovery. 7, 2003–2030.

[22] Jacopo Soldani and Antonio Brogi. 2021. Anomaly detection and failure root cause analysis in (micro)service-based cloud applications: a survey. (2021). https://arxiv.org/abs/2105.12378 arXiv: 2105.12378 [cs.SE].

[23] Jacopo Soldani, Damian Andrew Tamburri, and Willem-Jan Van Den Heuvel. 2018. The pains and gains of microservices: a systematic grey literature review. *Journal of Systems and Software*, 146, 215–232. https://www.sciencedirect.com/science/article/pii/S0164121218302139.

[24] Peter Spirtes, Clark Glymour, and Richard Scheines. 1993. *Causation, Prediction, and Search*. Vol. 81. (Jan. 1993). ISBN: 978-1-4612-7650-0.

[25] Richard S. Sutton, Francis Bach, and Andrew G. Barto. 2018. *Reinforcement learning: An introduction*. MIT Press Ltd.

[26] Ping Wang, Jingmin Xu, Meng Ma, Weilan Lin, Disheng Pan, Yuan Wang, and Pengfei Chen. 2018. Cloudranger: root cause identification for cloud native systems. In *2018 18th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID)*, 492–502. doi:10.1109/CCGRID.2018.00076.

[27] Li Wu, Johan Tordsson, Jasmin Bogatinovski, Erik Elmroth, and Odej Kao. 2021. Microdiag: fine-grained performance diagnosis for microservice systems. In *2021 IEEE/ACM International Workshop on Cloud Intelligence (CloudIntelligence)*, 31–36. doi:10.1109/CloudIntelligence52565.2021.00015.

[28] Li Wu, Johan Tordsson, Erik Elmroth, and Odej Kao. 2020. Microrca: root cause localization of performance issues in microservices. In *NOMS 2020 - 2020 IEEE/IFIP Network Operations and Management Symposium*, 1–9. doi:10.1109/NOMS47738.2020.9110353.

[29] Ruyue Xin, Peng Chen, and Zhiming Zhao. 2023. Causalrca: causal inference based precise fine-grained root cause localization for microservice applications. *Journal of Systems and Software*, 203, (Sept. 2023), 111724. doi:10.1016/j.jss.2023.111724.